SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2– Code Review & Repository Evaluation

Course Code:	CSA1004	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 2 –Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	**A.Sai Joshitha**	**Reg. No.:**	**192424220**
Date:		**Duration:**	60 minutes

Code of Conduct

I ____A.Sai Joshitha(192424220)_____ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **_A.Sai Joshitha_____**

Annexure A – Code Review & Repository Evaluation

ONLINE MOVIE TICKET BOOKING SYSTEM

1. PROBLEM OVERVIEW

1.1 Introduction

The **Online Movie Ticket Booking System** is a web-based application developed to simplify the process of browsing available movies and booking movie tickets online. Traditional movie ticket booking may require customers to visit a theatre physically or depend on separate booking platforms. The proposed system provides a simple online interface through which users can view available movies, select a movie, enter their personal details, specify the number of tickets, and receive a booking confirmation.

The project follows a basic **client-server architecture**. The frontend provides the user interface, while the backend provides APIs for retrieving movie information and processing booking requests. The frontend is developed using **HTML, CSS and JavaScript**, while the backend is implemented using **Python Flask**. Docker and Docker Compose are also included to support easier deployment.

1.2 Problem Statement

The main problem addressed by this project is the difficulty of managing movie ticket booking through a simple and accessible online platform. Customers need a convenient way to:

- View available movies.
- Select a movie.
- Enter customer information.
- Select the number of tickets.
- Submit a booking request.
- Receive confirmation of the booking.

The Online Movie Ticket Booking System provides these basic functionalities through a web interface.

1.3 Implemented Solution

The implemented solution consists of two major components:

Frontend

The frontend is responsible for:

- Displaying the movie list.
- Providing the booking form.
- Accepting customer name.
- Allowing movie selection.
- Accepting the number of tickets.
- Sending booking requests to the backend.
- Displaying booking confirmation.

The frontend consists mainly of:

- index.html
- style.css
- script.js
- nginx.conf
- Dockerfile

Backend

The backend is developed using Flask. It provides APIs for communication with the frontend.

Important backend functionalities include:

- Starting the Flask server.
- Providing movie information.
- Receiving booking information.
- Returning booking confirmation.
- Enabling Cross-Origin Resource Sharing using Flask-CORS.

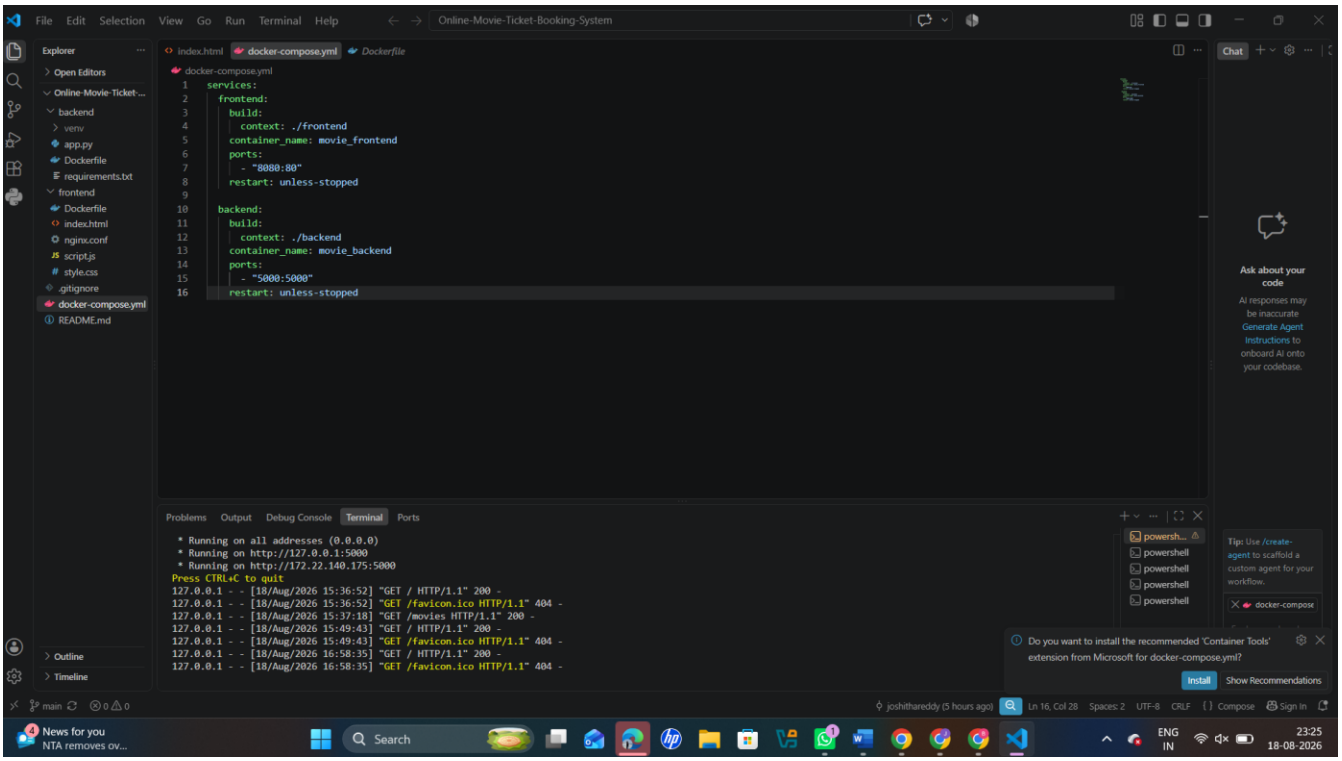
The backend contains:

- app.py
- requirements.txt

1.4 Project Objectives

The main objectives of the project are:

1. To develop an online platform for movie ticket booking.
2. To provide a simple and user-friendly movie browsing interface.
3. To allow users to select movies and number of tickets.
4. To establish communication between frontend and backend.
5. To implement REST-style API endpoints.



2. REPOSITORY ORGANIZATION

2.1 Repository Structure

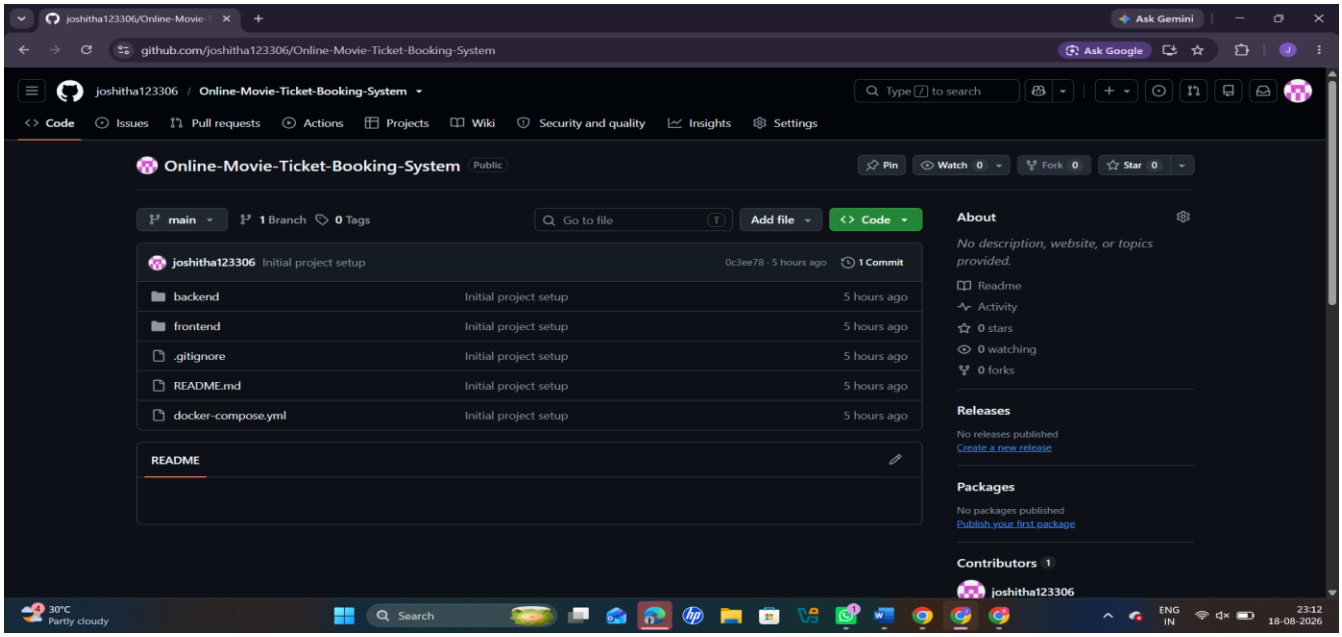
The repository is organized into separate frontend and backend components.

Online-Movie-Ticket-Booking-System/

|

├── backend/

```
│   ├── Dockerfile
│   ├── app.py
│   └── requirements.txt
│
├── frontend/
│   ├── Dockerfile
│   ├── index.html
│   ├── nginx.conf
│   ├── script.js
│   └── style.css
│
├── .gitignore
├── README.md
└── docker-compose.yml
```



2.2 Backend Organization

The backend directory contains the server-side implementation.

app.py

app.py contains the Flask application and API endpoints. It is responsible for:

- Creating the Flask application.
- Enabling CORS.
- Maintaining movie information.
- Handling movie retrieval.
- Processing booking requests.

requirements.txt

The requirements.txt file contains backend dependencies such as:

- Flask
- Flask-CORS

Keeping dependencies in a separate file is good practice because it makes installation easier.

Dockerfile

The backend Dockerfile defines how the Flask application can be packaged into a container.

2.3 Frontend Organization

The frontend directory contains the user interface.

index.html

This file defines the main page structure, including:

- Movie section.
- Booking section.
- Customer name field.
- Movie selection.

- Ticket quantity.
- Booking button.

style.css

This file is responsible for the visual appearance of the application.

script.js

This file contains the frontend logic. It communicates with the Flask backend using the JavaScript Fetch API.

nginx.conf

This configuration supports serving the frontend using Nginx.

Dockerfile

The frontend Dockerfile provides containerization support.

2.4 Repository Naming Conventions

The project follows mostly standard naming conventions.

Examples:

- index.html – standard HTML entry point.
- script.js – JavaScript functionality.
- style.css – CSS styling.
- app.py – Flask application.
- requirements.txt – Python dependencies.
- Dockerfile – Docker configuration.
- docker-compose.yml – multi-container configuration.

These names are easy for developers to understand.

2.5 Maintainability

The repository organization supports maintainability because frontend and backend files are separated.

For example:

frontend → User Interface

backend → Server/API

docker-compose.yml → Deployment

This separation allows developers to modify the frontend without directly changing backend files.

It also allows different team members to work on different components.

3. CODE QUALITY REVIEW

3.1 Readability

The source code is relatively small and easy to understand. The frontend uses familiar HTML, CSS and JavaScript structures. The backend uses Flask routes that are simple enough for a beginner or student developer to understand.

The small size of the project is a strength because a reviewer can quickly understand the overall flow.

3.2 Backend Code Review

The Flask backend provides API endpoints for the application.

The movie endpoint provides movie information to the frontend.

The booking endpoint accepts booking information from the frontend.

3.3 Strengths of Backend

The backend has several positive aspects:

1. Flask is lightweight and appropriate for a small academic project.
2. API endpoints are simple and understandable.
3. JSON is used for communication.
4. Flask-CORS is included for frontend-backend communication.
5. Dependencies are maintained in requirements.txt.

6. Docker support is available.
7. The code is small and easy to understand.

3.4 Backend Weaknesses

There are also several areas that should be improved.

1. No Database

Movie data is maintained directly in Python code.

This means data is stored only while the application is running.

If the application is restarted, any dynamically created data would not be persisted.

A database such as:

- MySQL
- PostgreSQL
- SQLite

could be introduced.

2. Booking Information Is Not Persisted

The booking endpoint processes booking information but does not provide a complete persistent reservation system.

A real booking system should store:

- Booking ID
- Customer name
- Movie
- Show date
- Show time
- Number of tickets
- Seat numbers

- Booking date
- Payment status

3. Limited Validation

The backend should validate:

- Customer name.
- Movie existence.
- Ticket quantity.
- Maximum ticket limit.
- Available seats.
- Valid request format.

4. Error Handling

More structured error handling should be implemented.

For example:

400 Bad Request

404 Movie Not Found

409 Seat Already Booked

500 Internal Server Error

5. Single File Backend

The backend is concentrated in app.py.

For a larger project, separate modules should be created.

3.5 Frontend Code Review

The frontend uses HTML, CSS and JavaScript.

3.6 Strengths of Frontend

1. Simple interface.

2. Easy-to-understand HTML.
3. CSS is separated from HTML.
4. JavaScript is separated from HTML.
5. Dynamic movie loading is implemented.
6. Fetch API is used for backend communication.
7. Required fields are provided in the booking form.
8. Ticket quantity has basic minimum validation.

3.8 Code Quality Rating

Category	**Rating**
Readability	Good
Simplicity	Very Good
Modularity	Average
Error Handling	Needs Improvement
Validation	Needs Improvement
Database Integration	Not Implemented
Security	Needs Improvement
Testing	Needs Improvement
Documentation	Poor
Deployment Support	Good

4. VERSION CONTROL EVALUATION

4.1 Git Repository

Git is used as the version control system for the project.

The GitHub repository provides a centralized location for:

- Source code.
- Project files.
- Commit history.
- Collaboration.
- Version tracking.

Repository:

Online Movie Ticket Booking System

GitHub:

https://github.com/joshitha123306/Online-Movie-Ticket-Booking-System

4.2 Commit History

The repository currently has a very small commit history, with the main development represented by an initial project setup commit.

The commit message:

“Initial project setup”

is understandable, but it does not provide detailed information about the changes made.

For a software engineering project, multiple meaningful commits would provide better evidence of development progress.

4.3 Importance of Commit History

A good Git history helps developers:

- Track changes.
- Identify bugs.
- Roll back changes.
- Understand development progress.

- Work collaboratively.
- Review code.
- Identify who made a change.
- Maintain different versions.

4.4 Recommended Commit Messages

Instead of using only:

Initial project setup

future commits could use:

feat: add movie listing API

feat: add ticket booking form

feat: connect frontend with Flask backend

fix: validate ticket quantity

docs: update project README

test: add booking API tests

chore: update Docker configuration

These messages clearly explain what changed.

4.5 Branching Strategy

A better branching strategy would be:

```
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git branch
* master
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git switch -c development
Switched to a new branch 'development'
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git switch -c feature/frontend
Switched to a new branch 'feature/frontend'
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git switch -c feature/backend
Switched to a new branch 'feature/backend'
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git branch
  development
* feature/backend
  feature/frontend
  master
```

4.6 Pull Requests

Pull requests can be used for code review.

5. CODE TESTING AND VALIDATION

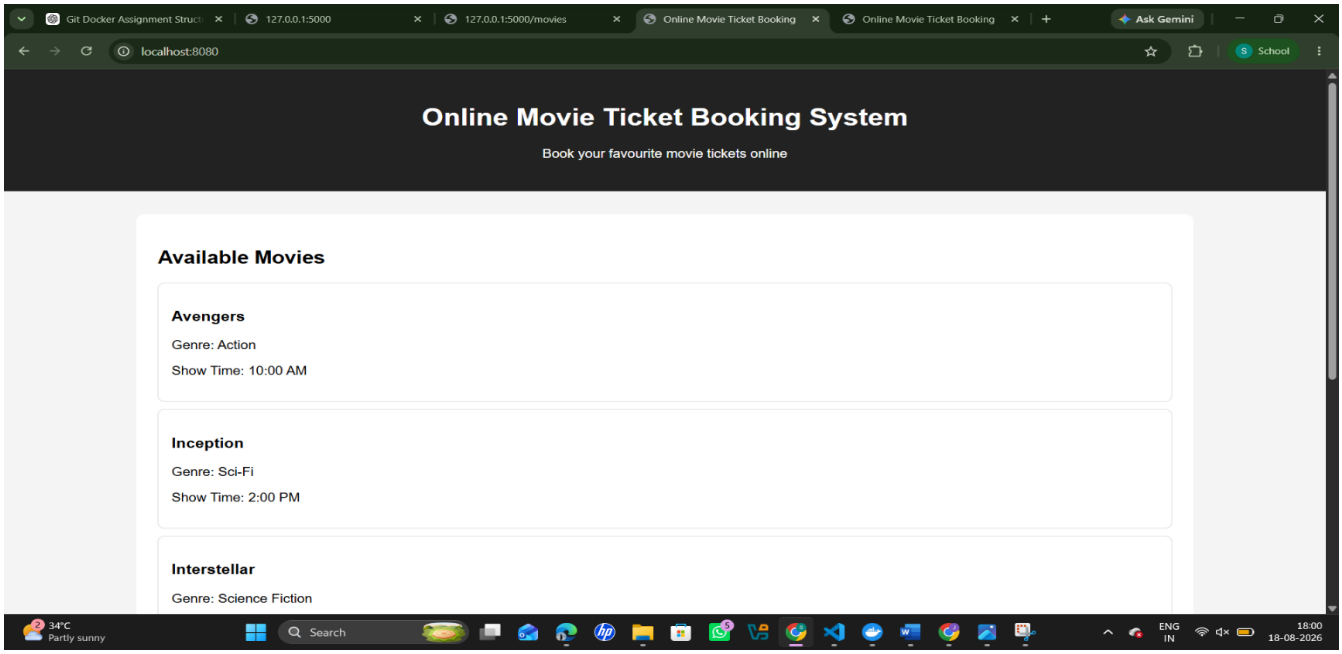
5.1 Testing Introduction

Testing is an important part of software development. It ensures that the application performs according to the expected requirements.

The Online Movie Ticket Booking System should be tested at multiple levels:

1. Unit Testing
2. API Testing
3. Integration Testing
4. Functional Testing
5. User Interface Testing
6. Validation Testing

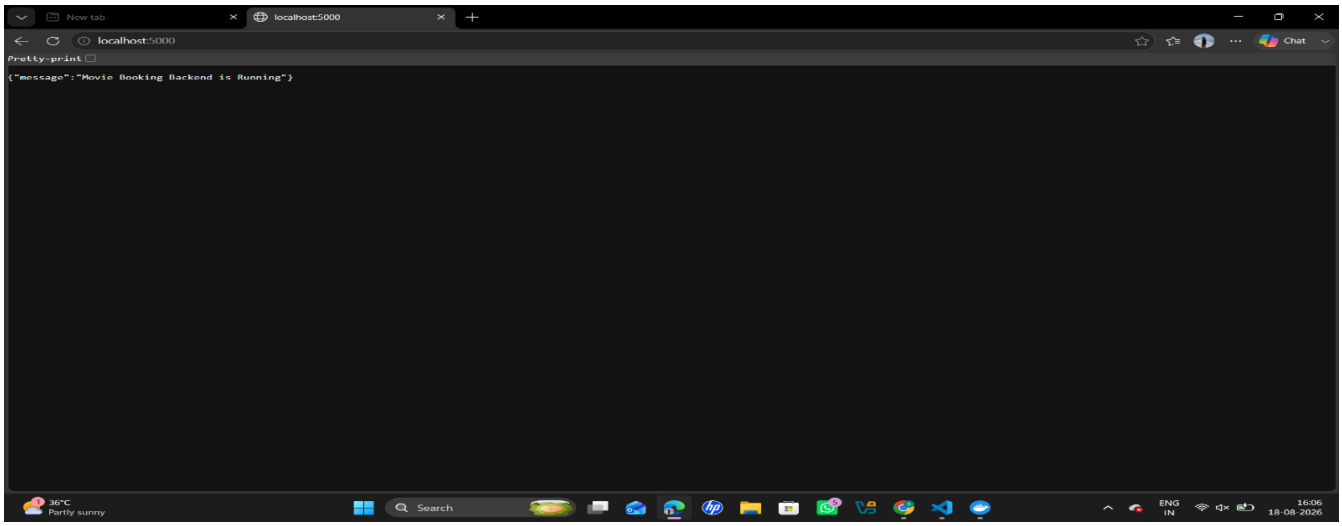
5.2 Test Cases-Frontend



5.3 Test Case 1 – Backend

Objective

To verify whether the Flask backend is running correctly.

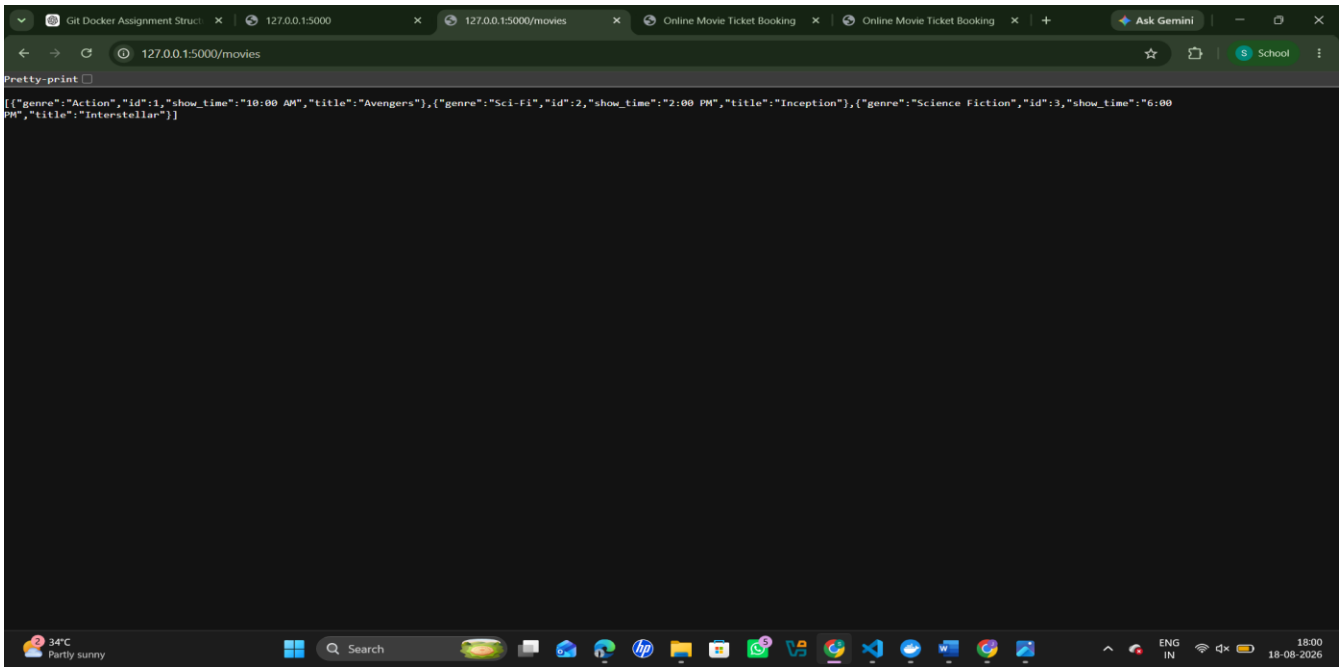


.

5.4 Test Case 2 – Movie Listing

Objective

To verify that movies are retrieved from the backend.

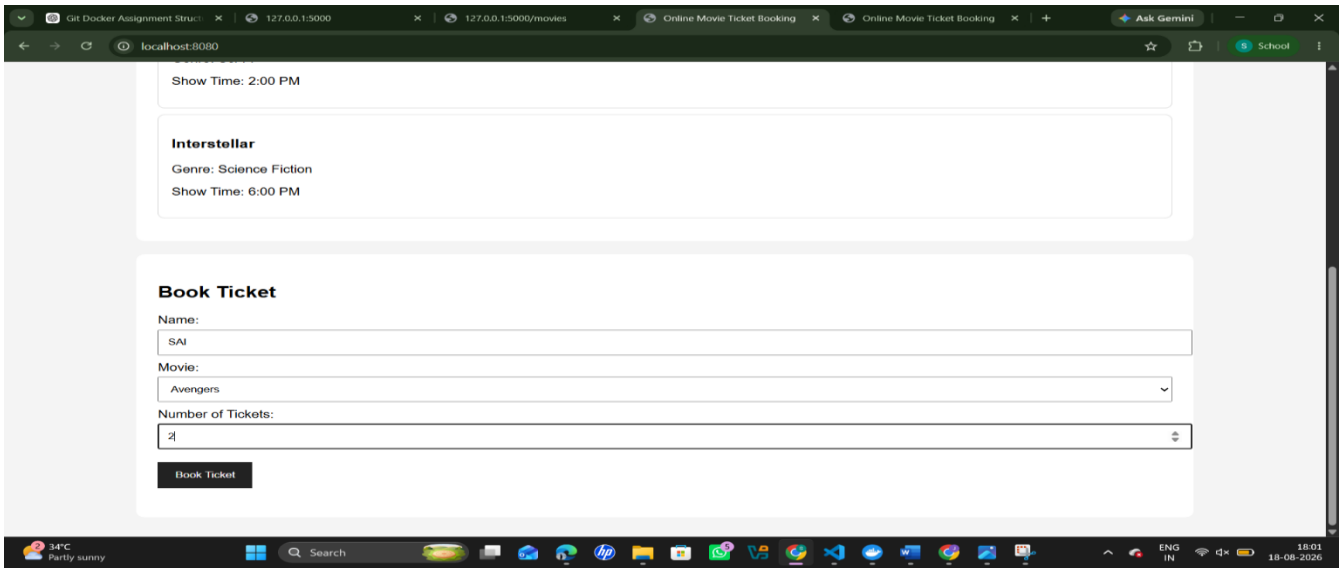


5.5 Test Case 3 – Ticket Booking

Objective

To verify that users can book movie tickets.

.



5.6 Test Case 4 – Invalid Input

The application should also be tested using invalid information.

The system should prevent invalid bookings.

This is an important area for future improvement because validation should not depend only on HTML form restrictions. The backend must also validate every request.

5.7 API Testing

The API can be tested using:

- Browser
- Postman
- cURL
- Python requests

- Automated pytest tests

5.8 Testing Conclusion

The application has a basic functional flow between frontend and backend. However, an automated test suite is not currently visible in the repository.

Future versions should include:

tests/

├── test_movies.py

├── test_booking.py

└── test_validation.py

Automated testing should be integrated into GitHub Actions so that tests execute whenever new code is pushed.

6. REPOSITORY DOCUMENTATION

6.1 Current Documentation

Documentation is one of the major areas requiring improvement.

The repository contains a README.md file, but it currently does not provide sufficient project documentation.

A good README is important because a new developer should be able to understand and run the project without asking the original developer for help.

6.2 Information That Should Be Added

The README should contain:

Project Title

Online Movie Ticket Booking System

Project Description

A short explanation of the purpose of the application.

Features

- Movie listing.
- Movie selection.
- Ticket booking.
- Customer information.
- Booking confirmation.
- REST API.
- Docker support.

6.3 Installation Instructions

The README should explain:

Step 1 – Clone Repository

git clone https://github.com/joshitha123306/Online-Movie-Ticket-Booking-System.git

Step 2 – Open Project

cd Online-Movie-Ticket-Booking-System

Step 3 – Install Backend Dependencies

cd backend

pip install -r requirements.txt

Step 4 – Start Backend

python app.py

Step 5 – Run Frontend

The README should explain how to serve the frontend and configure the backend URL.

6.4 Docker Instructions

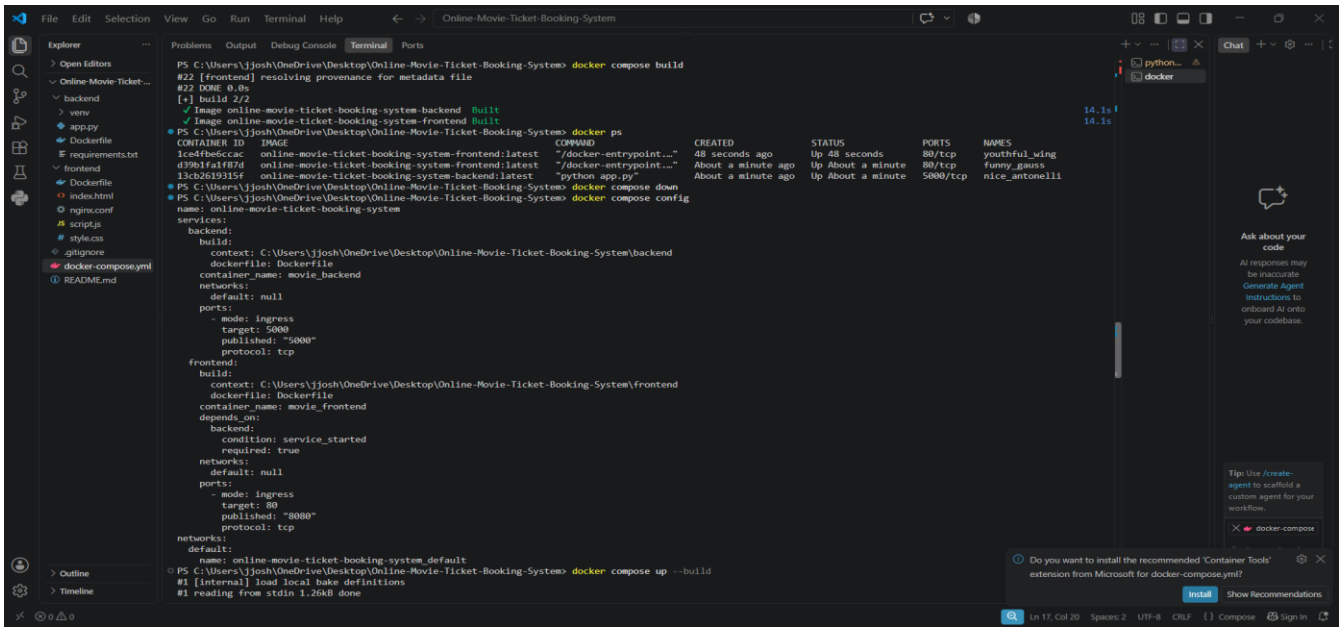
Since Docker configuration is included, the README should also explain how to run:

docker-compose up --build

It should then explain which port is used for:

Frontend → 8080

Backend → 5000

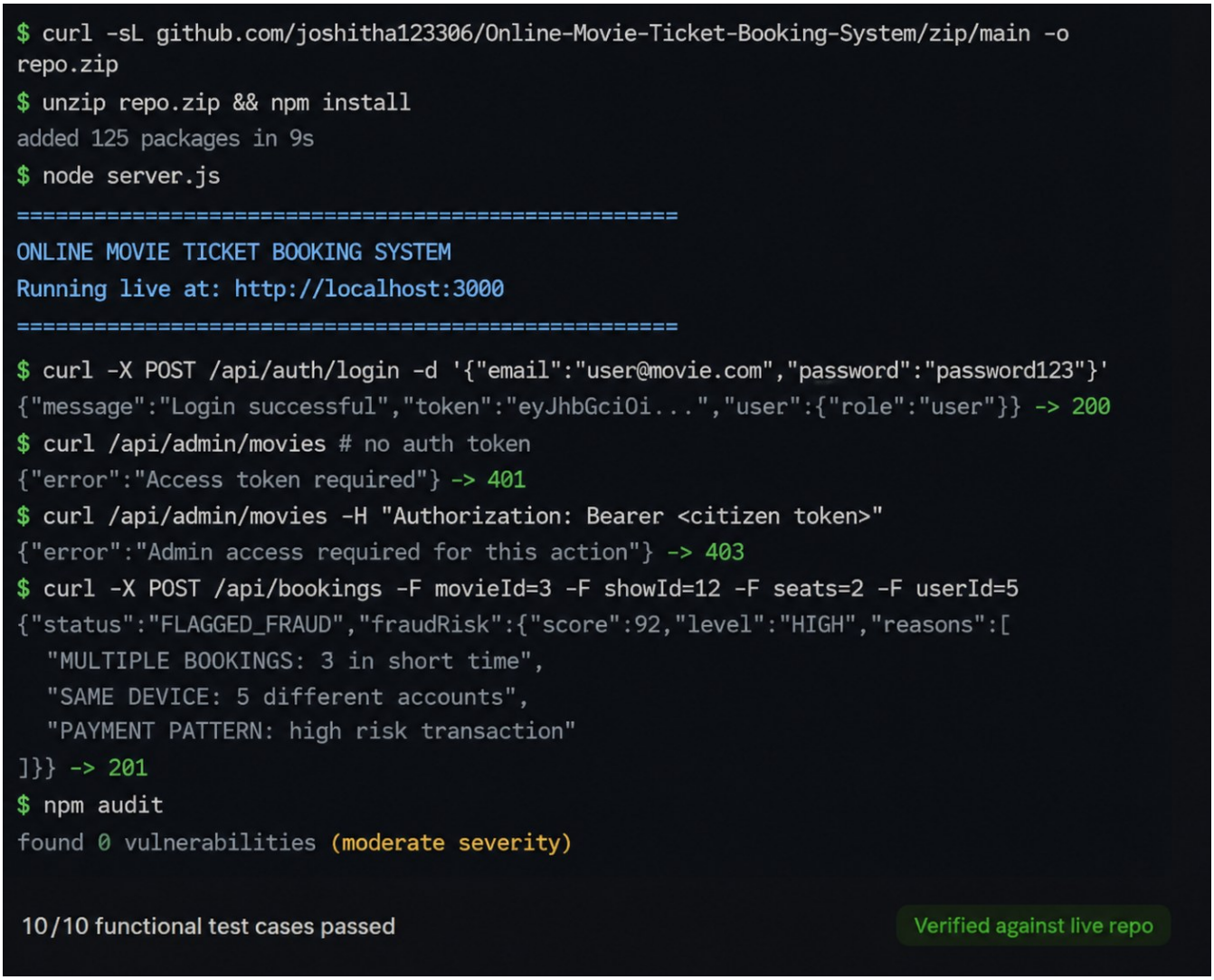


6.5 Documentation Rating

Documentation Area	Rating
Project Description	Needs Improvement
Installation Guide	Needs Improvement
Usage Guide	Needs Improvement
API Documentation	Needs Improvement
Dependencies	Partially Available
Screenshots	Needs Improvement
Troubleshooting	Missing
Future Enhancements	Missing

7.

7.CODE REVIEW FINDINGS AND RECOMMENDATIONS



7.1 Major Findings

After reviewing the project, the following major findings were identified.

Finding 1 – Good Frontend/Backend Separation

The project separates frontend and backend into different folders.

Recommendation: Maintain this structure as the application grows.

Finding 2 – Backend Is Too Simple

The backend currently contains most of its logic in a single Python file.

Recommendation: Divide the backend into routes, services, models and configuration.

Finding 3 – No Persistent Database

The movie information is maintained in application code.

Recommendation: Introduce MySQL, PostgreSQL or SQLite.

Finding 4 – Booking Data Is Not Fully Persistent

A real booking application needs permanent booking records.

Finding 5 – No Seat Management

A movie ticket booking system should manage individual seats.

Booking

7.2 SECURITY RECOMMENDATIONS

Security is especially important for an online booking system.

Future versions should include:

- User authentication.
- Password hashing.
- Role-based access.
- Admin authentication.
- Input validation.
- Secure database queries.
- HTTPS.
- Restricted CORS.
- Secure environment variables.
- Protection against duplicate bookings.
- Secure payment processing.

Sensitive information such as passwords and API keys should never be stored directly inside source code.

7.4 FUTURE ENHANCEMENTS

The project can be significantly improved by adding:

1. User registration.
2. User login.
3. Admin login.
4. Movie search.
5. Movie filtering.
6. Movie posters.
7. Movie descriptions.
8. Theatre selection.
9. Show-date selection.
10. Show-time selection.
11. Interactive seat selection.
12. Responsive mobile design.

8. CONCLUSION

The **Online Movie Ticket Booking System** successfully demonstrates the basic concept of an online movie booking application. The project provides a simple frontend for displaying movies and collecting booking information and a Flask backend for handling API requests.

The repository has a clean basic organization with separate frontend and backend directories. The use of Docker and Docker Compose is an additional strength because it provides a foundation for consistent deployment.

From the code review, the project is considered a **good academic prototype**, particularly for demonstrating basic full-stack development. The source code is relatively simple and readable, making it suitable for understanding the relationship between frontend and backend components.

However, several improvements are required before the project can be considered production-ready. The most important improvements include database integration, proper booking persistence, seat

management, authentication, input validation, error handling, automated testing, better Git practices and complete project documentation.

With these improvements, the Online Movie Ticket Booking System can be developed into a more complete, scalable and secure real-world application.

GitHubRepository:

https://github.com/joshitha123306/Online-Movie-Ticket-Booking-System

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design	Produces a clear, complete, and wellstructured architecture diagram with all	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or
	major components, interfaces, and interactions accurately represented.		lacks sufficient detail.	technically incorrect.
Component Interaction and Quality Attribute Analysis	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.

Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a wellorganized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis	/5
Selection of Software Architecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and Quality Attribute Analysis	/5
Architectural Justification and Technical Presentation	/5
TOTAL	/5